

Проектирую и реализую сложные программные системы на C++ и .NET — от compiler/runtime infrastructure и статического анализа до backend/service systems. Сильнее всего интересуют архитектурные задачи, где важны ясные границы компонентов, корректность, восстановление после сбоев и проверяемые инварианты.

Системная инженерия
C++/.NET: compiler/runtime infrastructure, статический анализ, backend/service systems и low-level codegen.

Архитектурная ответственность
UniversalToolchain: единый semantic planner и immutable plan; VpnMediator: explicit state/recovery; x86-64 backend: independent verifier + differential execution.

Профессиональная глубина
МЦСТ: LLVM 22/C++23 · ИСП РАН: статический анализ C#/.NET.

ОПЫТ

- 2026 — сейчас

ИСП РАН — инженер по статическому анализу
 - Участвую в разработке SharpChecker — платформы статического анализа C#/.NET в ИСП РАН.
 - Работаю над задачами символьных вычислений в контексте статического анализа C#/.NET.
- июль — август 2026

МЦСТ — стажёр по разработке компиляторов
 - Реализовал LICM-проход для LLVM 22 на C++; перенос выполняется только при выполнении условий по структуре цикла, обращениям к памяти, побочным эффектам и безопасности спекулятивного выполнения.
 - Собрал Python-обвязку для сравнения поведения программы до/после оптимизации и проверки случаев с преобразованием и без него.

ПРОЕКТЫ

UniversalToolchain / Wist2
Единственный публичный semantic planner разрешает зависимости и конфликты, выбирает providers, фиксирует порядок contributions и типизированные backend routes в одном immutable LanguagePlan до запуска runtime.
Архитектурные решения защищены pre-execution validation, exact manifest/ownership checks и parity interpreter/CIL; текущий exact manifest — 1 324/1 324.

VpnMediator
Явные payment/access states отделяют приём внешних событий от идемпотентного reconciliation и recovery; миграции и operational gates делают lifecycle-переходы проверяемыми.
Recovery включает backup/restore, health-gated переключение release и rollback; публичный кейс не раскрывает секреты и пользовательские данные.

x86-64 Compiler Backend & Register Allocation
Стратегия register allocation отделена от независимого assignment verifier до phi lowering и генерации SysV x86-64.
Семантика сгенерированного native-кода проверяется differential execution относительно эталонного интерпретатора; тесты покрывают ветвления, циклы, spills и parallel phi moves.

КОМПЕТЕНЦИИ

Software / Systems
C++23, C#/.NET, Python, Linux, backend/runtime infrastructure, low-level/system components

Архитектура
ownership/module boundaries, typed contracts, immutable planning, explicit state machines, lifecycle/recovery

Compiler / Program Analysis
LLVM 22/IR, CFG/SSA, .NET/CIL, static/data-flow analysis, optimization legality

Reliability / Verification
idempotency, reconciliation, backup/restore, regression tests, independent verifiers, differential execution

ОБРАЗОВАНИЕ

НИУ ВШЭ — Программная инженерия

ДОСТИЖЕНИЯ

UniversalToolchain / Wist2 — один из победителей конкурса проектов с открытым кодом студентов ФКН.

Москва / удалённо